

Marcia-Automated-Workflow-for-Protein-Ligand-LIE-Linear-Interaction-Energy-Calculations: An End-to-End Q6 Application for Protein–Ligand LIE Simulations

Marcia Yineth Castillo,[†] Gian Pietro Miscione,[†] and Jordi Villà i Freixa^{*,‡}

[†]*Departamento de Química, Universidad de los Andes, Bogotá 111711, Colombia*

[‡]*Universitat de Vic – Universitat Central de Catalunya, Vic, Barcelona 08500, Spain*

E-mail: jordi.villa@uvic.cat

Abstract

Linear interaction energy (LIE) calculations are attractive for binding-affinity estimation because they offer a practical compromise between simulation cost and interpretability, but routine application still requires a multi-step workflow that is often assembled manually from loosely connected scripts. We present Marcia-Automated-Workflow-for-Protein-Ligand-LIE-Linear-Interaction-Energy-Calculations, an end-to-end Q6-based application workflow that integrates ligand preparation, protein–ligand complex preparation, equilibration, replicated production simulations, trajectory-quality control based on interaction-energy stability, automated extension of insufficiently converged simulations, LIE free-energy evaluation, and downstream regression analysis. The workflow is organized into repository modules for ligand-in-water and complex-in-water calculations and provides Python and shell utilities for generating Q6 input files, producing topology- and FEP-compatible systems, scanning production log files

for Q-surr. electrostatic and van der Waals energy drift, flagging simulations with errors above 1 kcal/mol, and exporting ligand-wise ΔG estimates to CSV files. A key usability feature is that stability assessment is treated as a first-class stage before free-energy reporting, reducing the risk of accepting underconverged production runs. The software is designed as an automation layer around Q6 and QLigFEP rather than as a replacement for those packages. This application note describes the architecture of the workflow, the scientific and technical advances it introduces for routine LIE studies, and the practical requirements for reproducible deployment on cluster environments.

1 Introduction

The linear interaction energy (LIE) method estimates ligand binding free energies from averaged electrostatic and van der Waals interaction terms obtained from end-state simulations.^{1,2} Its appeal in drug-discovery settings comes from a favorable balance between computational cost, physical transparency, and compatibility with ligand-series studies. In practice, however, a publishable LIE study depends on more than the final equation. Users must prepare ligand and complex systems consistently, generate input files for multiple simulation stages, manage replicated molecular dynamics runs, verify that the relevant interaction energies are stable enough for analysis, and only then compute and compare ΔG values.

Q6 provides the simulation engine needed for LIE and related free-energy calculations,³ while QLigFEP provides a route to ligand parameterization and related setup tasks. What is often missing is a coherent application layer that joins these capabilities into a workflow that can be followed, audited, and scaled across multiple ligands. Many groups still rely on local notes, one-off scripts, and manually edited inputs, which makes reproducibility fragile and greatly increases the effort required to extend a study from a single ligand to a series.

Marcia-Automated-Workflow-for-Protein-Ligand-LIE-Linear-Interaction-Energy-Calculations was developed to address this practical gap. The repository is organized as a complete protocol rather than a single script. It includes ligand and complex preparation modules, equi-

libration and production templates, analysis utilities to inspect Q-surr. interaction energies, automated generation of extended production inputs when convergence criteria are not met, LIE calculation scripts, and regression-analysis tools for reference ligands. The central contribution of the application is therefore workflow integration: it turns a scattered sequence of Q6-dependent tasks into a repeatable protocol for protein–ligand LIE calculations.

2 Workflow Overview and Implementation

The repository is structured around two main simulation branches, `ligands/` and `complex/`, reflecting the requirement that LIE be evaluated from separate simulations of ligand-in-water and protein–ligand complex-in-water systems. Each branch contains staged preparation and simulation logic, while the repository root provides cross-cutting analysis scripts for quality control, LIE calculation, and regression-based benchmarking.

In the ligand branch, the workflow starts from numbered ligand PDB files and generates ligand-specific library, parameter, topology, and FEP files. The documentation indicates use of QLigFEP-related tools and auxiliary scripts such as `bind_prm.py`, `copy_generate.py`, and `generate_ligand_fep.py` to produce the assets needed for Q6 execution. Equilibration inputs are generated automatically and a three-stage equilibration protocol yields restart files for production. Production input files are then created and executed as two replica-like runs per ligand.

In the complex branch, the workflow concatenates a prepared protein structure with each ligand, builds Q6 preparation inputs for the corresponding protein–ligand complex, prepares the solvated complex system, creates a one-state FEP mapping for the ligand atoms in the complex, and then runs analogous equilibration and replicated production stages. Complex-specific helper scripts include `concat_protein_ligands.py`, `generate_complex.py`, `generate_eq_inp.sh`, and `generate_inp_files.py`.

Figure 1 summarizes the full application path. After ligand and complex production

runs are available, the workflow inspects Q-surr. electrostatic and van der Waals energies from the production log files. The scripts `ligand-surrounding-energies.py` and `check_high_errors.py` implement a simple but operationally valuable stability criterion: the trajectory is split into two halves, and the error estimate is defined as half the absolute difference between the average interaction energies of the two halves. Simulations with either electrostatic or van der Waals errors greater than 1 kcal/mol are flagged for extension. This criterion follows the practical guidance used in LIE applications.²

If the threshold is not met, `prueba_2.py` generates extended production input files and corresponding Slurm launch scripts under `check_errors/`, allowing the user to continue the MD runs without rebuilding the whole system. This feature is a meaningful technical advance for day-to-day use because it turns convergence checking from a manual decision into an explicit, scripted branch of the workflow.

Once the production logs satisfy the stability requirement, `analyze_LIE_noqgui.py` parses ligand and complex log files through the shared `mdlog_energies.py` module and computes the LIE estimate

$$\Delta G_{\text{bind}} = \beta \left[\left\langle V_{\text{el}}^{\text{complex, wat}} + V_{\text{el}}^{\text{complex, prot}} \right\rangle - \left\langle V_{\text{el}}^{\text{lig, wat}} \right\rangle \right] + \alpha \left[\left\langle V_{\text{vdW}}^{\text{complex, wat}} + V_{\text{vdW}}^{\text{complex, prot}} \right\rangle - \left\langle V_{\text{vdW}}^{\text{lig, wat}} \right\rangle \right] + \gamma \left[\left\langle V_{\text{vdW}}^{\text{complex, prot}} \right\rangle - \left\langle V_{\text{vdW}}^{\text{lig, wat}} \right\rangle \right] \quad (1)$$

from the extracted Q-protein and Q-water interaction terms. The root-level Slurm wrapper `analyze_LIE_ligands.sh` loops across ligand indices, dispatches the calculation for each ligand pair, and writes CSV summaries to `results/`. In the current repository, this launcher exposes the empirical coefficients directly in the shell script, with example values $\alpha = 0.68$, $\beta = 0.11$, and $\gamma = 0.0$, making calibration choices transparent to the user.

The repository also includes downstream evaluation utilities such as `DG_regression_reference_ligand` which fits a linear regression between experimental and calculated free energies and reports metrics including R^2 , MAE, and RMSE. This places the workflow beyond mere simulation

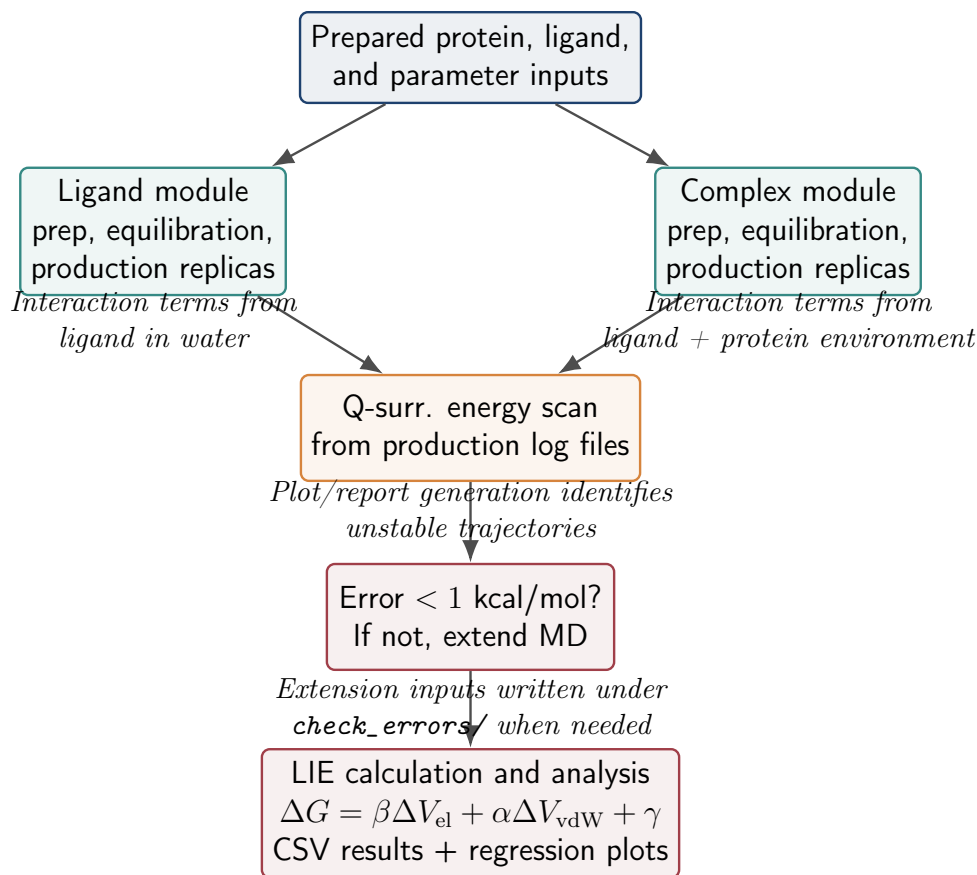


Figure 1: General scheme of the Marcia-Automated-Workflow-for-Protein-Ligand-LIE-Linear-Interaction-Energy-Calculations application. The workflow prepares ligand and complex systems separately, runs replicated Q6 equilibration and production simulations, checks Q-surr. interaction-energy stability, branches to MD extension when the error threshold is not met, and finally computes LIE binding free energies and downstream analyses.

setup by supplying an explicit postprocessing path for reference-ligand validation.

3 Usability Advances

The principal advance of this application is that it formalizes the entire LIE protocol, not only the final energy calculation. The repository provides explicit module boundaries for ligand preparation, complex preparation, equilibration, production, quality control, free-energy estimation, and regression analysis. This reduces hidden know-how and makes the protocol easier to hand off between users.

The second major advance is the integration of quality control into the default workflow. Many practical LIE pipelines compute ΔG directly from whatever production trajectory is available. Here, the scripts explicitly inspect Q-surr. interaction energies, generate per-log plots, identify simulations that exceed an error threshold, and support extension of the simulation length before free-energy reporting. This is a scientifically important usability feature because it encourages users to reject unstable trajectories rather than accept them silently.

Third, the workflow is cluster-oriented and scriptable. Slurm launchers are supplied for preparation, production, extension, and LIE evaluation, which makes the application suitable for routine academic HPC use. The output is machine-readable, primarily as CSV tables and report files, so the calculated energies can be integrated directly into later benchmarking or medicinal-chemistry analysis steps.

Finally, the software exposes empirical choices instead of burying them. Simulation lengths, coefficient values, and directory expectations are visible in the scripts and tutorials, which makes the protocol easier to audit and adapt. For a JCIM Application Note, this combination of automation, traceability, and reviewer-testable behavior is more important than claiming methodological novelty.

4 Demonstration Case in the Current Repository

The target repository already contains a worked example spanning ligand and complex setup files, production logs, quality-control plots, an error report, and at least one LIE result CSV file. It also contains a reference-ligand regression plot for an MDM2 inhibitor series. This is a stronger basis for an application note than the previously used analysis-only repository because it shows the end-to-end protocol in a single codebase.

For the final journal submission, the demonstration should be framed as an end-to-end MDM2 inhibitor workflow: start from a prepared protein and ligand structures, generate the

ligand and complex Q6 systems, run equilibration and replicated production, inspect the Q-surr. interaction-energy stability, extend simulations if needed, compute ligand-wise LIE ΔG values, and compare selected reference ligands against experiment. The current repository contents already support that narrative and provide concrete filenames and outputs that reviewers can inspect.

5 Data and Software Availability

The software described here should be made available through a public versioned repository under the name used in the manuscript title, together with a tagged release archived through a persistent repository service. The release should include the ligand and complex tutorials, the helper scripts required for input generation and analysis, representative example files, and a reviewer-accessible minimal test case. Because the workflow depends on Q6 and QLigFEP, those dependencies should be cited and linked separately.³⁻⁵

The current implementation assumes an HPC-style environment with Python 3.8 and Slurm available for execution of the supplied batch scripts. Some preparation steps also rely on external software in the ligand module. The final manuscript should state clearly which components are operating-system agnostic and which are expected to run in a Linux cluster setting. That distinction is important for compliance with JCIM software-availability expectations.

6 Associated Content

Supporting Information. Suggested Supporting Information for the final submission includes a reviewer test protocol, a minimal end-to-end example for one ligand, the directory schema expected by the analysis scripts, and a compact table mapping each workflow stage to its inputs and outputs.

7 Author Information

Corresponding Author

Jordi Villà i Freixa, Universitat de Vic – Universitat Central de Catalunya, Vic, Barcelona 08500, Spain; Email: jordi.villa@uvic.cat

Notes

The authors declare no competing financial interest.

8 Acknowledgments

This draft reserves space for funding, institutional computing resources, and any acknowledgments related to Q6, QLigFEP, and the preparation of the MDM2 ligand series used as the demonstration case.

References

- (1) Åqvist, J.; Marelus, J. The Linear Interaction Energy Method for Predicting Ligand Binding Free Energies. *Comb. Chem. High Throughput Screen.* **2001**, *4*, 613–626.
- (2) Gutiérrez-de Terán, H.; Åqvist, J. *Computational Drug Discovery and Design*; Humana Press, 2012; pp 305–323.
- (3) Bauer, P.; Barrozo, A.; Purg, M.; Amrein, B. A.; Esguerra, M.; Wilson, P. B.; Major, D. T.; Åqvist, J.; Kamerlin, S. C. L. Q6: A Comprehensive Toolkit for Empirical Valence Bond and Related Free Energy Calculations. *SoftwareX* **2018**, *7*, 388–395.
- (4) Q Users Q6 Repository – EVB, FEP and LIE Simulator. <https://github.com/qusers/Q6>, 2026; Accessed 2026-03-30.
- (5) Q Users QLigFEP Repository. <https://github.com/qusers/qligfep>, 2026; Accessed 2026-03-30.

TOC Graphic

